IEM 4723 Information Systems Design
Assignment 3: NoSQL and MySQL

Some learning objectives for you to consider as you attempt these questions.

1. The first objective of this assignment is for you to understand the handling of a Redis DB instance. This means you need to be able to understand and reason with respect to all the Redis commands and directives we covered in class.
2. The second objective of this assignment is to be able to draw a distinction with respect to implementation, data processing and querying capabilities of NoSQL and SQL paradigms.
3. The third objective is for you to familiarize yourself with the unique capabilities that Redis offers with respect to handling IoT sensor datasets. For this I have included a question that uses a simple dataset pertaining to an industrial reactor. You can find more details about this dataset **here** if youre interested. You do not need to understand the dataset to attempt these questions. Only the ability to read the data, insert it into Redis and be able to query it is required which would require you to create your own conventions of how to store and process keys.

Before You Begin

1. Bring up the Redis stack in `docker_tutorial/redis-dev/` and the MySQL stack in `docker_tutorial/mysql-dev/`. Both must be running to answer these questions. You might need to adjust their ports to make sure they dont clash within the `docker_compose.yml` file. Copy the supplied `data/` folder where your notebook can reach it.
2. You are only allowed to use the Redis commands that have been discussed in class. Consequently, you must also be able to defend your usage of whatever commands you have used in your submission.

Part 1: Load the Data [40 points]

1. [**15 points**] Create a MySQL database named `redis_compare_<yourCWID>`. Load `customer` from `Customer.csv` and `customerorder` from `CustomerOrder.csv`, both in `data/orders/`. Declare `Customer_Num` and `Order_Num` as primary keys and `Customer_Num` in `customerorder` as a foreign key. Choose your own column types. You are welcome to reuse code that you developed for Assignment 2. You do not need to apply the key suffix from Assignment 2. Report the row count of each table.
2. [**25 points**] In Redis, load the same two files so that the following objectives are accomplished:
   - one hash per customer where each customer can be uniquely identified.
   - one hash per order where each order can be uniquely identified.
   - one set per customer, `orders:customer:<Customer_Num>`, holding that customer's order numbers.
   - a set that can hold all customer identities.
   - a set of all customers that can hold every customer who has placed a valid order.

Part 2: Comparison with SQL [30 points]

For each question, write the SQL against your Part 1 database and the redis-py against your Part 1 keys, and show both outputs.

1. **[10 points]** Which city does customer `C25` live in?
2. **[10 points]** How many orders has customer `C25` placed?
3. **[10 points]** Which customers live in Pennsylvania? Report how many customer hashes your Redis solution had to read.

Part 3: Reactor Telemetry [30 points]

`data/reactor_sample_5k.csv` holds 4,901 readings from 27 reactor runs. `Timestamp_min` is unique across the file and increasing. Columns are described in `data/README.md`. You are to utilize Redis commands for answering each questions and must minimize the use of Python loops. Use the function contained in `data/load_reactor.py`, to read the data; you do not need to write your own reader.

1. **[5 points]** Load the sensor data into Redis where each data record (i.e. the per-minute row in the CSV) can be uniquely identified. You can use one hash per record to do this.
2. **[5 points]** Create one sorted set per run, `run:<Reactor_Run_ID>:temperature`, whose members are that run's `Timestamp_min` values, each scored by the temperature recorded at that timestamp.
3. **[5 points]** Using these sorted sets, report the top 3 temperatures for each run together with the timestamp at which each one occurred.
4. **[5 points]** For run 4, report the temperature recorded at timestamp 500, and the rank of that timestamp within run 4's temperature sorted set.
5. **[5 points]** Create one sorted set per run, `run:<Reactor_Run_ID>:pressure`, whose members are that run's `Timestamp_min` values, each scored by the pressure recorded at that timestamp.
6. **[5 points]** Report the maximum pressure recorded in each run, together with the timestamp at which it occurred, using these sorted sets.

Deliverables

Submit the following through Canvas. Do not submit your work to the course GitHub repository.

1. A Jupyter notebook named `assignment3_<yourCWID>.ipynb`, with every part in order under a markdown heading per part, saved with its output visible.
2. Begin Part 1 and Part 3 with the appropriate `flushdb()` so that a re-run does not inherit keys from an earlier attempt.

Grading

This assignment is graded out of 100 points as shown above, which accounts for 60% of your grade for this assignment. The remaining 40% comes from a short, three-question viva, where you will

be asked to explain specific choices in your submission, such as why a particular Redis structure was the right one and what assumptions you made.

Data

Reactor telemetry: *Batch Reactor Anomaly Data (Sample)* by AI Mind Teams, Kaggle, `https://www.kaggle.com/datasets/aimindteams/batch-reactor-anomaly-data-sample`, used under CC BY-NC 4.0.